MASTER IMPLEMENTATION BRIEF

MANYA ECOSYSTEM

An evidence-based execution brief for the implementation agent: what the Manya and Manya-OS repositories verifiably contain today, the two-system federation architecture, and the phased mandate for shared foundations, Cortex knowledge sharing, Aruk, and the HAEL Foundation domain.

HAEL Foundation

Prepared for the implementation agent - full build mandate

OCTOBER 2026 · BASELINE: MANYA + MANYA-OS

Table of Contents

1. Executive Summary and Mandate 3

1.1 The recorded architecture decision . . . 3

1.2 Critical distinctions the agent must not blur . . . 3

2. Verified Baseline: The Manya Repository 4

2.1 What is genuinely implemented . . . 4

2.2 Package and tool inventory . . . 5

2.3 Lycon, Aruk, and the Foundation in this tree . . . 5

2.4 Defect register . . . 6

3. Verified Baseline: The Manya-OS Repository 7

3.1 The governed runtime slice . . . 7

3.2 Package inventory . . . 7

3.3 The keyring is real cryptography with a federation-shaped hole . . . 8

3.4 The persistence backbone that nobody uses . . . 9

3.5 Seeds of the vision found in the tree . . . 9

3.6 Hygiene issues in this repository . . . 9

4. Target Architecture: Two Manyas and the Controlled Bridge 10

4.1 Responsibility contracts . . . 11

4.2 Why a bridge rather than one runtime . . . 11

4.3 The visibility iron rule . . . 11

5. Shared Foundations: Transport, Persistence, Wiring 12

5.1 Remote transport for the nervous system . . . 12

5.2 Durable persistence behind the runtimes . . . 13

5.3 Package strategy across two repositories . . . 13

5.4 Wiring targets . . . 13

6. Cortex Reference-Based Knowledge Sharing 14

6.1 The KnowledgeRecord and its reference . . . 14

6.2 The registry contract . . . 15

6.3 What must not change and how to prove it . . . 15

7. Security Model: Aruk and the Trust Boundaries **16**

7.1 Responsibilities . . . 16

7.2 Mapping existing primitives to Aruk . . . 17

7.3 Trust boundaries and failure posture . . . 17

8. The HAEL Foundation Domain (Manya-Foundation) **18**

8.1 Identity tiers and the permission matrix . . . 18

8.2 Platform capabilities . . . 19

8.3 The live relationship map, done correctly . . . 19

9. Phased Execution Roadmap **19**

10. Test Plan, Risks, and Open Decisions **20**

10.1 Verification matrix . . . 20

10.2 Risk register . . . 21

10.3 Open decisions the agent must resolve with evidence . . . 22

1. Executive Summary and Mandate

This brief is the working contract between the HAEL Foundation and the implementation agent. It converts the Manya vision - a connective nervous system for applications, devices, and intelligences, extended by a people-and-organization platform for the Foundation - into an executable program across two existing repositories. Every claim about current capability in this document was verified by direct inspection of the uploaded repository snapshots (`Manya-main` and `Manya-OS-main`), not inherited from the repositories' own documentation, which is partly stale. Where a statement rests on a repository document rather than on executable code, that dependence is called out explicitly.

The mandate is a **full build mandate**: the agent is expected to audit, repair, extend, integrate, and test across both repositories, not to return another architecture proposal. The division of responsibility is that this document fixes the destination, the boundaries, the security principles, and the acceptance criteria, while the agent owns the code-level decisions inside those constraints. Three outcomes define success: a networked connective fabric that works across machines rather than only inside one process; a Cortex layer in which knowledge is shared by reference under authorization rather than copied; and a separate Foundation-facing Manya system serving the HAEL Foundation's people and organizations, federated with the technical Manya through a controlled bridge.

1.1 The recorded architecture decision

The vision holder has decided the question that the earlier analysis left open: **the Foundation-facing Manya is a second, distinct Manya system**, not an application bolted onto the existing one. It communicates with the original Manya through controlled connections. This decision, referred to throughout as the "two-Manyas" model, has three consequences. First, the existing Manya repository remains the technical connectivity domain (Domain A) and is not reshaped into a member platform. Second, a new system, provisionally Manya-Foundation (Domain B), is built on the shared foundations that both systems import. Third, every interaction between the domains crosses a Federation Bridge that enforces identity, consent, and policy on both sides. Section 4 specifies the resulting architecture; Section 8 specifies Domain B itself.

6 / 20	0	2,151	15
Manya-OS packages actually wired into its runtime	occurrences of "Aruk" in either repository	tests passing per the Manya-OS closure document	SQL tables in the persistence backbone, unwired

1.2 Critical distinctions the agent must not blur

> **Two category errors to avoid**
>
> First, Manya's event dashboard (`site/manya/public/manya-live.html` and the federated-identity graph page) is a technical visualization of the event bus. It is **not** the HAEL Foundation platform, and building member features into it would repeat the category error this brief exists to prevent. Second, Cortex in both repositories is a task-dispatch and reasoning fabric (`packages/cortex/src/index.js` is 198 lines in Manya; Manya-OS adds a heuristic planner). That is **not** the reference-based knowledge-sharing behavior specified in Section 6, although Manya-OS's KnowledgeRegistry is the seed it will be grown from. Treating either of these as "already done" is the single most likely way to ship the wrong system.

Finally, a note on evidence discipline. Several repository documents state things the code no longer supports: Manya's README and SKILL.md still describe a Lycon browser tool that is absent from the tree, claim 1,715 tests where the cross-tool matrix file now contains two, and a release closure whose certified runtime no longer parses. The agent should treat executed tests and read code as the only sources of truth, and should fix the documents as part of Phase 0 rather than trusting them.

2. Verified Baseline: The Manya Repository

Manya (the `Daemon22/Manya` snapshot) is a real and substantial codebase: 19 packages, 15 tools, a CLI with an HTTP+SSE server, a gateway runtime, and 46 test files. Its self-description - "everything connected, everyone unified" - is accurate in one narrow sense: within a single Node process, it connects tools, identities, and events well. What it does not yet do is cross a machine boundary. No package in the tree opens a socket for fabric purposes; both the unify event bus and the nervous-system fabric advertise themselves as local-only in their own READMEs.

2.1 What is genuinely implemented

The connective core is the `tools/unify` package (`@manya/unify`), and its four services are real code with 75 tests, not documentation. The tool mesh (`src/mesh.js`) registers tools with manifests and routes 58 capabilities to their owning tools through `packages/toolkit`'s capability-owner map. Identity federation (`src/federation.js`) creates, links, resolves, and merges typed identifiers such as ORCID, DOI, email, and HS codes in an in-memory store. The event bus (`src/eventbus.js`) publishes, subscribes, routes by declared sync channels, and replays history - synchronously and in-process. Vocabularies (`src/vocabularies.js`) translate between static maps, for example HS chapters to industries. On top of this sits a 7-tool CLI registry (`tools/cli/src/registry.js`) with an HTTP and SSE server (`src/serve.js`), plus hand-rolled canvas visualizers that render the mesh and identity graph.

Around the core sit 11 composed packages - keyring, attest, ledger, anonymize, memory, cortex, perception, telepathy, reflection, economy, and guardian - each of which composes the tools into a capability (sovereign keys, device attestation, hash-chained audit, signed inter-agent messaging, and so on). All 11 are implemented and unit-tested, and none is imported by the runtime, the CLI, or each other in production paths; they are libraries awaiting wiring. Separately, six TypeScript packages merged from

Manya-OS (constitution, contracts, council, customs-shield, nervous-system, weave) carry 670 vitest specs that now run green, and they too have zero runtime consumers in this repository.

2.2 Package and tool inventory

Component	What inspection found	Status
tools/unify (mesh, federation, event bus, vocabularies)	Real, 75 tests; in-memory and single-process; no transport of any kind	Core, local-only
packages/toolkit	Capability-owner map (58 entries), 15 tool manifests; no lycon or hawk manifest	Core, declarative
11 composed packages (keyring, attest, ledger, memory, cortex, telepathy, ...)	Implemented and tested; no production consumers anywhere in the tree	Orphaned libraries
6 merged TS packages (constitution, contracts, council, customs-shield, nervous-system, weave)	Substantial TS with 670 vitest specs passing; committed dist; no runtime consumers in this repo	Merged, unwired
15 tools under tools/	All implemented with node:test suites; only 7 are CLI-dispatchable; upmp is Python (4,035 lines) plus a JS adapter with no bridge	Mixed
runtime/index.mjs gateway	Intended as HTTP gateway proxying /api/os/* to Manya-OS; currently fails to parse	Broken
runtime/connector.mjs	Real MCP-over-HTTP client, GitHub read-only connector, capability gate, in-memory or Supabase store	Real, isolated
site/manya + dashboards	Marketing site; event-bus dashboard and identity-graph page fed by the CLI SSE stream	Technical-only

Table 1: Manya repository inventory with verified status.

2.3 Lycon, Aruk, and the Foundation in this tree

A case-insensitive search for "Aruk" across code, docs, and markdown returns **zero occurrences**. If Aruk is to be the security and authorization authority of the ecosystem - and Section 7 assumes it is - it is greenfield work, informed but not constrained by existing primitives. "Lycon" is the opposite case: roughly thirty references across README, SKILL.md, the architecture audit, the license text, and the worklog describe a Lycon browser integration with seven `lycon:*` sync channels, a `window.__lyconNative` bridge contract, and a `manya browse` command - but the directory `tools/lycon-browser/` does not exist in this snapshot, no manifest declares it, and the CLI registry holds seven tools, not eight. Lycon is an external repository whose integration here was removed without updating the prose; the documented channel and bridge contracts are nonetheless useful prior art for

Section 4's external interface.

The HAEL Foundation appears as steward and brand: authorship in `package.json`, a five-repo ecosystem model in the README (Manya, MANYA Intelligence OS, Gqobonco, OMNIMIND, Orren), a canon document (`docs/canon/HAEL-CODEX-ASSET-ASSIGNMENTS.md`) that assigns Manya the "connective and orchestration layer" role, and a "Living Tree" metaphor in the audit report. There is no member, partner, profile, directory, or announcement feature anywhere in the tree; the words "council" and "member" belong to the AI consensus package and to RBAC test fixtures respectively. Domain B has no existence in this repository, which the two-Manyas decision now makes a feature rather than a gap.

2.4 Defect register

Defect	Evidence	Impact on the mandate
Gateway runtime cannot start	runtime/index.mjs raises SyntaxError "Missing catch or finally after try" at line 36	The only cross-repo surface is dead; e2e suite cannot run; Phase 0 must repair it first
Missing build script for merged packages	Six Apache-2.0 packages call node ../../scripts/build-package.js; scripts/ does not exist	Package rebuilds fail; only committed dist/ artifacts work
Documentation drift	README/SKILL.md/audit report reference lycon-browser, 1,715 tests, a 162-test matrix; the tree has none of these	Agent guidance would be corrupted if trusted; docs must be reconciled in Phase 0
Version drift	Root package.json says 1.0.0; CLI prints 0.9.0; docs say 0.9.0	Release engineering and dependency pinning become unreliable
Cross-tool matrix collapsed	tests/performance-7x7.test.js is 23 lines with 2 tests; docs describe 162	Integration coverage across tools is effectively absent
Identity capability unused	Federation registry exists but no runtime path issues or checks credentials	Domain A has no authentic identity; Section 7 builds on it

Table 2: Manya defect register, verified by inspection.

3. Verified Baseline: The Manya-OS Repository

Manya-OS (the `manyahael/intelligence-os` snapshot) is the systematic counterpart: 20 package directories of which 13 TypeScript packages build and test coherently, a small governed HTTP runtime, and the only real persistence backbone in the ecosystem. Its closure document (`RELEASE-CANDIDATE-CLOSURE.md`, September 2026) is the most candid status artifact in either repository and is largely consistent with inspection: 87 suites and 2,151 tests passing, typecheck and 13/13 builds clean, and an explicit admission that the runtime keeps ledger and memory in memory. Seven plain-JS packages ported from Manya sit in the tree broken and unimported.

3.1 The governed runtime slice

The runtime (`runtime/index.mjs`) is a plain `node:http` server bound to `127.0.0.1:3200` that refuses non-loopback binds without a 32-character bearer token, rate-limits to 120 requests per minute, caps bodies at 1 MiB, and validates every request body against schemas compiled by `@manya-os/contracts` (`runtime/api-contract.mjs`). Endpoints cover health, runtime description, events, memory recall and remember, reasoning, ledger, and governance. Every call passes through the constitution package's EnforcementEngine, and denials return 403 with an audit identifier that is written to the ledger. This is a genuine governance slice: authorization is not decorative here. Its limitations are equally clear - the runtime imports exactly 6 of 20 packages (cortex, memory, nervous-system, ledger, constitution, contracts), mints throwaway identities of the form `manya-os:` instead of using the keyring, and holds all state in process memory.

3.2 Package inventory

Package	Verified capability	Runtime wiring
keyring	AES-256-GCM, RSA-PSS/ECDSA P-256, did:key identities, W3C-style verifiable credentials, role manager, deny-by-default access enforcer, capability grants with revocation, Shamir recovery, signed multi-device sync bundles	Not wired
attest	Device fingerprinting, signed challenge-response with single-use nonces, sessions with revocation, weighted trust evaluator, remote attestation quotes	Not wired
ledger	Hash-chained events, RFC 6962-style Merkle tree with proofs, timestamp authority, replayer, file store	Wired (in-memory)

Package	Verified capability	Runtime wiring
constitution	Policy condition language (no eval), RBAC with inheritance, decision hierarchy, emergency controller, enforcement engine with grant registration and revocation	Wired (PDP)
contracts	Schema compilation, manifest validation, semver compatibility diffing, API contract checking	Wired
cortex	Goal decomposition, topological planner, tool registry, keyword router, scheduler, retries, KnowledgeRegistry provenance and delta tracking	Wired
memory	Five store types, TF-IDF index, per-record permissions, snapshot delta sync, shareable CollaborationPackages, "Hub" doc-contract	Wired (in-memory)
nervous-system	In-process event fabric: compiled filters, priority router, bounded queues, ring recorder, metrics; CollaborationRequestQueue; five hardware sources are stubs	Wired (local)
supabase	Postgres/Supabase adapters for ledger, memory, keyring storage, attest sessions; migration runner; 15-table schema with RLS	Not wired
council, weave, anonymize, customs-shield	Deterministic consensus engine; graph model with layouts and SVG export; PII redaction; sanctions and HS screening	Not wired
7 ported JS packages	Manya composed packages copied in; they import @manya-os/vault, shield, unify, lens, hawk which do not exist here, and call Manya-style memory APIs	Broken

Table 3: Manya-OS packages - capability and wiring status.

3.3 The keyring is real cryptography with a federation-shaped hole

The keyring package (4,387 lines, 11 spec files) deserves its own paragraph because most of the security architecture in this brief stands on it. Identities are real did:key derivations from RSA-PSS 3072 or ECDSA P-256 keys; credentials are W3C-style verifiable credentials with canonical serialization and genuine signatures; wallet export uses PBKDF2-SHA512 at 210,000 iterations wrapped around AES-256-GCM; recovery implements Shamir secret sharing over GF(2^8) with Lagrange interpolation. Capability grants support issuer, holder, resource wildcards, expiry, use limits, and revocation records. Two gaps matter. Revocation state lives only inside the issuing manager instance, so a remote verifier has no way to learn that a credential died - there is no distributed revocation registry. And the signed sync bundles, which would otherwise be the natural federation primitive, deliberately specify out-of-band transport: there is no network code in the package at all. Closing both gaps is Aruk's job (Section 7), not a reason to rewrite the keyring.

3.4 The persistence backbone that nobody uses

The supabase package is the only code in either repository that talks to anything beyond the process: adapters implementing the ledger, memory, keyring-storage, and session interfaces on Postgres, a numbered migration runner with a direct-`pg` executor and RPC fallback, and six migration files defining 15 tables - ledger events, four memory stores and links and permissions, attest sessions, four keyring tables, council debates and decisions, constitution audit, customs reports, plus connector connections - with row-level security policies and roughly 25 indexes (`packages/supabase/migrations/001_initial_schema.sql` onward). It is tested by 11 unit and 12 environment-gated integration specs. Nothing in the runtime imports it. The backbone is therefore best understood as purchased land: the schema, security model, and adapters exist, and wiring them into the runtime is Phase 1 work measured in days, not weeks.

3.5 Seeds of the vision found in the tree

> **Build on these three seeds**
>
> Three implemented primitives map directly onto the vision and must be extended, not replaced. (1) Cortex's `registry/knowledge.ts` tracks which component owns which knowledge key, refuses owner conflicts, supports transfer, and computes deltas so "only changed data is transferred" - it is intra-instance, but it is the provenance spine of reference-based sharing. (2) Memory's CollaborationPackage exports only records marked shareable and its documentation defines a "Hub" contract that "never exposes the full memory" - the Hub itself does not exist. (3) Nervous-system's CollaborationRequestQueue buffers bounded, TTL-limited inter-instance collaboration requests locally. Each is one step short of the networked behavior the vision requires.

3.6 Hygiene issues in this repository

Four hygiene problems need Phase 0 attention. The seven ported JS packages cannot resolve their own dependencies and call APIs this repo's TypeScript packages do not export; they must be either repaired against local interfaces or quarantined out of the workspace, and the README currently omits them entirely. The changelog claims 12 packages against 13 real ones and 20 directories, and carries a future-dated entry, which is a small thing that corrodes trust in the release process. The verification claims rest on a closure document plus committed artifacts because `node_modules` is absent from the snapshot - the agent must rerun the full suite and re-anchor those claims in observed runs. And the runtime governance file, while real, is only exercised by tests; production policy content for the Foundation does not exist yet.

4. Target Architecture: Two Manyas and the Controlled Bridge

The architecture separates concerns that the current tree keeps tangled: machine connectivity from human organization, and local coordination from cross-system trust. Domain A, Manya-Technical, is the existing repository continuing its role as the connective fabric for tools, devices, and intelligences, including the future Lycon host integration. Domain B, Manya-Foundation, is a new system that owns the HAEL Foundation's people and organizations: registries, profiles, announcements, communications, and shared resources. Between them stands a Federation Bridge that carries nothing without identity, consent, and policy. Two external systems complete the picture: Lycon, the browser and host platform, connects to Domain A through its documented channel contract; Aruk, the security and authorization authority, is trusted by both domains and by the bridge itself.

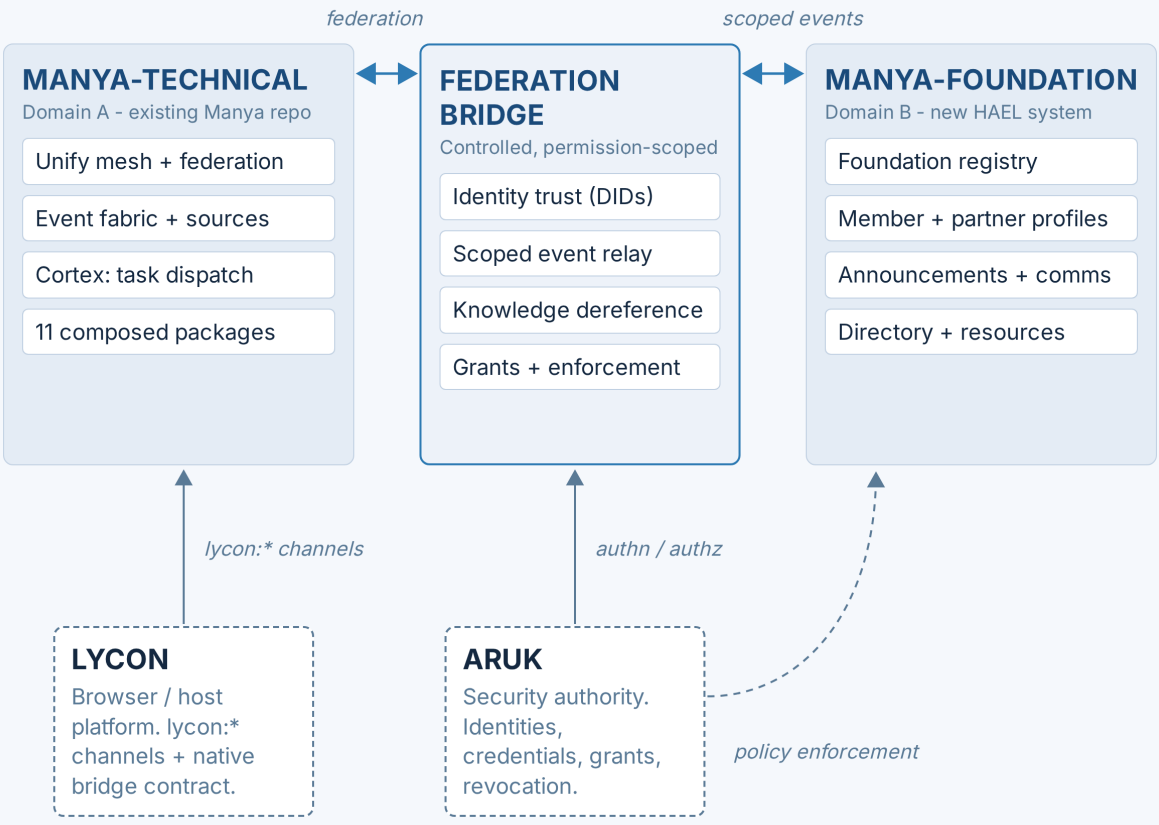


Figure 1: Two-Manyas target architecture with the controlled federation bridge and external systems Lycon and Aruk.

4.1 Responsibility contracts

Component	Owns	Must never do
Manya-Technical (Domain A)	Tool mesh, event fabric, device and application connectivity, Cortex dispatch, Lycon channel endpoints	Hold member personal data; trust cross-domain events without bridge verification
Manya-Foundation (Domain B)	Member/partner/visitor identities, profiles, directory, announcements, communications, shared resources, Foundation governance workflows	Expose raw internal events to Domain A; bypass Aruk for authorization decisions
Federation Bridge	Identity trust translation, scoped event relay, knowledge dereference proxy, grant enforcement, consent logging	Store copies of shared knowledge; widen scope beyond the presented grant
Lycon (external)	Browser/host runtime, lycon:* sync channels, window.__lyconNative bridge, local user consent UI	Assume Manya identity without Aruk-issued credentials
Aruk (external)	Identity registry, credential issuance and verification, capability grants, revocation, trust evaluation, policy decisions	Sit inside either Manya process; it must remain independently deployable

Table 4: Responsibility contracts for the five architectural actors.

4.2 Why a bridge rather than one runtime

The one-system alternative would put personal data and machine telemetry in one event fabric, one set of RBAC roles, and one blast radius. It would also couple the Foundation's release cadence to the technical platform's, and force a single identity model onto members and devices that want very different lifecycles. The two-system model instead makes trust an explicit, auditable hop: every cross-domain interaction is an opportunity to verify a credential, check a grant, and log consent. The cost is that the bridge must be engineered rather than assumed - which is why Sections 5 through 7 specify its transport, its knowledge contract, and its security model in that order. A seed already exists: Manya's gateway runtime was designed to proxy `/api/os/*` to Manya-OS, and its e2e suite expects the sibling checkout; the bridge generalizes exactly that relationship.

4.3 The visibility iron rule

> **Binding rule for every surface**
>
> Discovery, visibility, and access are three separate permissions, and conflating them is the architecture's cardinal security failure. A participant may be able to discover that an entity exists, without seeing its relationships; see a profile, without reading its knowledge; read a record, without learning that other readers exist. The connected-entity map (Section 8.3) must therefore evaluate the three permissions independently per edge and per node, default-deny each of them, and log every escalation. This rule binds Domain A, Domain B, the bridge, and every visualization built on top.

In practical terms this means the bridge never floods: event subscriptions across the boundary are granted per topic pattern, per subscriber identity, and per direction, with expiry. It means the graph visualizers in Domain A render only the requester's visible slice, computed by the same policy engine that gates reads. And it means Foundation member discovery cannot be scraped: directory listings are paginated, permission-filtered views, not raw table dumps. Any component that cannot answer "which grant allows this view?" must not ship.

5. Shared Foundations: Transport, Persistence, Wiring

Everything else in this brief stands on three foundations that do not exist yet: a remote transport for the event fabrics, durable persistence behind the runtimes, and a package strategy that stops the two repositories drifting apart as they share code. This section specifies all three. The guiding principle is to extend what inspection proved solid - the nervous-system fabric, the supabase adapters, the contracts system - rather than introducing new frameworks.

5.1 Remote transport for the nervous system

Both nervous-system packages and the unify event bus are explicitly in-process: no socket, no broker, no wire format. The mandate is to add a transport layer under the existing API, not to replace the API. The agent should introduce a transport interface with local, relay, and offline implementations: the local implementation delegates to today's fabric and preserves current semantics for every existing test; the relay implementation carries signed envelopes between Manya-Technical, the bridge, and Manya-Foundation over persistent connections; the offline implementation queues outbound envelopes durably and drains them on reconnect with the bridge acknowledging receipts. Envelopes must be authenticated end to end - sender DID, sequence number, timestamp, and signature - so the relay cannot forge, reorder undetected, or replay. Topic addressing should reuse the fabric's existing topic model, with the bridge translating between the two domains' vocabularies the way `tools/unify/src/vocabularies.js` already translates between maps.

Choose the wire protocol during Phase 1 and record the decision in the log (Section 10.3); a WebSocket relay with JSON or CBOR envelopes is the default recommendation because the runtime stack is already

`node:http`-based and the MCP connector (`runtime/connector.mjs`) proves the pattern works. Whatever is chosen, the transport must survive the three failure modes the vision names - online, offline, and reconnecting participants - and the queue semantics already implemented in the fabric (bounded queues, backpressure, drain) should be reused rather than reinvented.

5.2 Durable persistence behind the runtimes

The runtime in Manya-OS currently keeps its ledger and memory in process memory even though the supabase package implements the stores it needs. Phase 1 wires them: ledger events, memory stores, and connector connections move to the migration-managed Postgres schema, with the in-memory implementations retained for tests. The Supabase service-role key must remain server-side only, exactly as `DATABASE_BACKBONE.md` prescribes, with row-level security enabled on every member-facing table that Phase 4 adds. Manya-Technical's connector store already anticipates this posture (`createSupabaseConnectorStore` in `runtime/connector.mjs` persists to `manya_connector_connections`); completing the pattern there is small, bounded work. Durable persistence is what converts the event fabric from a telemetry curiosity into infrastructure that can replay, audit, and recover.

5.3 Package strategy across two repositories

Today the six Apache-2.0 packages and the seven composed packages exist in both repositories under different names and slightly different versions, and Manya-OS is already ahead (its nervous-system carries the CollaborationRequestQueue that Manya's copy lacks). Left alone, this fork drift becomes the ecosystem's chronic disease. The agent must pick and record one sharing strategy from: a single source-of-truth repository with versioned releases consumed by the other side; a synchronized subtree with a CI job that fails on divergence; or a build-time super-bundle split, which Manya-OS's `scripts/build-super-bundle.js` already demonstrates. Whatever is chosen, the seven broken JS ports in Manya-OS must be resolved first - repaired against local TypeScript interfaces, or removed from the workspace and documented - because they are the loudest existing evidence of drift. The 58-entry capability map in `packages/toolkit` should be extended with every new capability this mandate introduces, keeping "distinct product ownership" assertable.

5.4 Wiring targets

Target	Today	Required end state	Home
Nervous-system transport	In-process only, both repos	Signed envelope transport with relay and offline queue implementations behind the existing API	Both
Runtime persistence	In-memory ledger and memory	Supabase adapters wired; migrations applied on start; RLS verified	Manya-OS

Target	Today	Required end state	Home
Manya-Technical runtime	Broken gateway; connector isolated	Gateway repaired; connector wired to CLI registry and fabric; proxy to bridge	Manya
Composed packages	No production consumers	At minimum keyring, attest, ledger, memory, telepathy consumed by a running surface in each domain	Both
Identity across systems	Throwaway manya-os:uuid identities	Keyring did:key identities issued, stored, and verified on every cross-system call	Both + Aruk
Package sharing	Duplicated, drifting forks	One recorded strategy enforced by CI; broken JS ports resolved	Both

Table 5: Phase 1 wiring targets and their homes.

6. Cortex Reference-Based Knowledge Sharing

The vision's most precise requirement is that when one intelligence possesses a knowledge record, a second intelligence should discover that the record exists, check that it is authorized, and use the original through a controlled reference - not retrieve a copy into its own store. This is a different mechanism from shared memory, and inspection confirms it does not exist today: Manya's cortex is a 198-line dispatch fabric, and Manya-OS's KnowledgeRegistry tracks provenance only within one instance. This section specifies the missing mechanism. The distinction that drives the design is that **knowing where information exists** is a different permission from **being allowed to read it**, and both are different from **holding its content**.

6.1 The KnowledgeRecord and its reference

A KnowledgeRecord is registered once by its owning intelligence and is addressed by a stable, namespaced, content-addressed reference of the form `knowledge://owner-did/collection/id@version`. The record carries a MIME-typed payload or a locator to one, a schema tag validated through the existing contracts package, the owner identity, an availability mode (online, mirrored-read-only, or archived), integrity digest, and an access policy handle that delegates to Aruk. References are the only thing that ever crosses a trust boundary as a first-class object: they can be included in events, embedded in messages, stored in other intelligences' indexes, and logged, without transmitting the payload. Two references to the same record are the same knowledge - that is the property that makes "no copying" checkable.

6.2 The registry contract

Operation	Semantics	Fails when
register(record, policy)	Records owner, digest, schema, availability; returns canonical reference; indexes it for scoped discovery	Reference already owned by another identity (conflict, mirroring KnowledgeRegistry today)
lookup(criteria, requester)	Scoped discovery: returns references and metadata the requester may see - never payloads	Never fails closed; returns an empty, honest result set
authorize(ref, requester, action)	Asks Aruk whether read, list, or delegate is granted for this reference; result is time-boxed	Grant missing, expired, or revoked
deref(ref, requester, mode)	Controlled read: streams payload or view through the bridge after authorize(); supports full and partial modes	Authorization absent, owner offline and no mirror agreed, or digest mismatch
revoke(ref) / transfer(ref, newOwner)	Owner-initiated lifecycle; transfer mirrors existing KnowledgeRegistry.transfer semantics plus Aruk re-grant	Caller is not the current owner

Table 6: Knowledge registry API contract.

The registry is a capability of the bridge, with a local shadow in each domain: each Cortex keeps the KnowledgeRegistry it has, extended to store and honor foreign references it has been granted. Availability handling follows the record's mode - online records dereference through to the owner; mirrored records are readable from an agreed replica whose digest the registry re-verifies; archived records require the owner to revive them. Ownership conflicts reuse the existing "one owner per key" rule, and transfer re-issues grants under the new owner so that old references fail authorization rather than silently leaking.

6.3 What must not change and how to prove it

Nothing in this design asks the agent to weaken what already works: dispatch, planning, decision chains, and hash-chained auditing stay as they are; the registry is additive. The no-copy invariant is the acceptance criterion, and it is mechanically checkable: after an authorized access by intelligence B to a record owned by intelligence A, B's persistent stores must contain the reference and any derived annotations, but none of A's payload bytes, and the access must appear in both parties' ledgers with the grant identifier. Section 10.1 makes this a standing test, and Phase 2's exit depends on it passing across two processes, two identities, and one revoked case.

> **The no-copy invariant in one sentence**
>
> Discovery returns references; authorization gates dereferencing; ledgers record both. If a consumer can read a payload without a grant appearing in a ledger, the design has failed, regardless of how convenient the shortcut was.

7. Security Model: Aruk and the Trust Boundaries

Aruk is the ecosystem's security authority and, per the two-Manyas decision, an external system in its own right: independently deployable, trusted by Domain A, Domain B, and the bridge, and nameable in policy the way a certificate authority is nameable. Inspection showed that neither repository contains anything called Aruk, so this section defines it by responsibility and by the primitives it must stand on - all of which already exist and are decent quality. The agent's job is assembly and gap-closing, not cryptography.

7.1 Responsibilities

- **Identity registry.** Maintain DIDs for intelligences, people, devices, and organizations; bind them to keyring-verified public keys; record Foundation role bindings (member, partner, council, visitor) that Domain B consumes.
- **Credential lifecycle.** Issue and verify verifiable credentials through the keyring's existing issue/verify/validate paths; time-bound every credential; publish status so verifiers can check freshness.
- **Capability grants.** Extend the keyring grant manager into the ecosystem-wide grant language used by the bridge and the knowledge registry: who may act on which resource, from which domain, until when, and with what use limits.
- **Revocation.** Operate the distributed revocation registry that the keyring lacks: revocations must propagate to all trusting surfaces in bounded time, and verifiers must be able to ask "was this credential valid at time T?"
- **Trust evaluation.** Absorb the attest package's weighted trust evaluator and remote attestation quotes so that device posture can condition grants without leaking PII.
- **Policy decision.** Host constitution EnforcementEngine instances as the policy decision point for cross-domain requests; the bridge and both runtimes become enforcement points that cache decisions briefly and fail closed.

7.2 Mapping existing primitives to Aruk

Existing primitive	Aruk responsibility	Gap to close
keyring did:key identities and wallets	Identity registry entries	Networked registry and resolution API
keyring verifiable credentials	Credential issuance and verification	Status/refresh endpoint; issuer governance
keyring CapabilityGrantManager	Ecosystem grant language	Shared schema; cross-domain scoping; storage
In-memory grant revocation records	Distributed revocation	Distributed registry; propagation; time-T queries
attest TrustEvaluator and quotes	Device trust conditioning	Wire into grant decisions; privacy review
constitution EnforcementEngine and policy language	Policy decision point	Deploy as service; policy sets per domain
runtime bearer-token and rate-limit hardening	Transport security floor	Mutual TLS or signed envelopes at the bridge

Table 7: Existing security primitives and what Aruk adds.

7.3 Trust boundaries and failure posture

Four boundaries carry the model's weight, and each has a stated failure posture. Human-to-Foundation (Domain B logins): credentials verify against Aruk; a failed verification is a denied session, never a degraded one. Machine-to-machine inside a domain: requests carry credentials verified locally against cached trust data; if Aruk is unreachable, previously granted, unexpired access continues to work and new grants wait - availability must not require the authority to be online, but new trust must. Cross-domain (through the bridge): every envelope is verified by the bridge, every dereference is authorized, and nothing crosses without a ledger entry on both sides; the bridge fails closed. Device enrollment: attest challenge-response plus Aruk-issued device credentials, with the trust evaluator's thresholds deciding between trust, challenge, and reject exactly as implemented. Revocation is the sharpest edge of all: the Phase 3 exit test is an end-to-end scenario where a credential is revoked at Aruk and every surface, including the bridge and any offline replica, refuses it within the propagation budget the agent records.

8. The HAEL Foundation Domain (Manya-Foundation)

Domain B is the people-and-organization platform the vision describes: the place where Foundation members, council participants, partners, businesses, organizations, and the simply curious meet the ecosystem. It is a distinct Manya system built on the shared foundations of Section 5, the knowledge contract of Section 6, and the security model of Section 7 - not a fork of the technical dashboard, and not a bolt-on to Domain A. Its product surface is conventional and its guarantees are not: every capability below is permission-filtered per the visibility iron rule, every identity is an Aruk-verified identity, and every action lands in the ledger.

8.1 Identity tiers and the permission matrix

Four participant tiers cover the vision's distinction between recognized members and partners, visitors, and prospective participants. Tiers are claims issued by the Foundation (through Aruk) onto an underlying identity, so a person can hold different roles in different spaces without multiple accounts. The matrix below is the starting policy set for the constitution engine; the Foundation refines it over time without code changes.

Capability	Member	Partner	Visitor	Prospect
Create a profile in the directory	Yes	Yes (organization profile)	Scoped view only	Application profile
Discover other members	Yes	Within sector and shared interests	No	No
Read member-wide announcements	Yes	Yes	Public subset	Public subset
Open communications with a member	Yes	Yes, consent-gated	Via public channels only	No
Access shared resources	Per resource grant	Per resource grant	Published items	Onboarding set
Join council or sector spaces	As elected or invited	As contracted	No	No
See the relationship map	Own ego graph	Own ego graph	No	No

Table 8: Starting permission matrix across Foundation tiers.

8.2 Platform capabilities

- **Foundation registry.** The canonical store of people, organizations, sectors, and their relationships, with tier claims and consent records; schema-managed through the contracts package and persisted per Section 5.2.
- **Profiles.** Member and partner profiles with visibility-controlled fields; profile data is personal data, so retention and export obligations from the open decision in Section 10.3 apply from the first migration, not retrofitted.
- **Announcements.** Audience-addressed publication (all members, partners, public) with delivery receipts; publishing rights are grants, not roles hard-coded in the app.
- **Communications.** Consent-gated member-to-member and member-to-partner messaging reusing telepathy's signed-message patterns where machine participants are involved, and ordinary moderated channels where humans are.
- **Shared resources.** A grant-gated resource space whose items may be knowledge records (Section 6), files, or event streams; everything the bridge knows how to carry.
- **Onboarding.** The prospect journey - explore public material, apply, get verified, receive tier claims - modeled as a workflow in cortex with human approval steps for the Foundation's stewards.

8.3 The live relationship map, done correctly

The vision's "visual web of connected people, intelligences, and spaces" is the showcase surface, and it is also where the visibility rule earns its keep. Domain B should render each participant's ego graph - themselves and the nodes and edges they hold discovery or visibility grants for - using the weave package's deterministic layouts and SVG export rather than another hand-rolled canvas. Steward-facing views widen the slice under explicitly issued oversight grants. What no one ever gets is the whole graph: the full map exists only as a query-time union of permitted slices, and the bridge event relay must apply identical filtering so a determined participant cannot reconstruct hidden structure from event traffic. Manya's existing identity-graph visualizer demonstrates the rendering craft; its willingness to show everything is exactly what not to copy.

9. Phased Execution Roadmap

The phases follow the audit order the vision holder recommended - connective runtime first, then knowledge sharing, then security, then the Foundation domain, then production readiness - because each phase's exit criteria de-risk the next phase's assumptions. Phases 0 through 2 are almost entirely inside the existing repositories; Phase 3 stands up Aruk; Phase 4 builds Domain B; Phase 5 hardens the whole. The agent records per-phase evidence (test output, decision log entries, updated docs) as it goes, because the Foundation reviews at phase boundaries.

Phase	Objective and key work	Exit criteria
0 - Hygiene and repair	Fix runtime/index.mjs syntax error; restore merged-package builds; quarantine or fix the 7 broken JS ports; reconcile README/SKILL/audit drift; align versions; rerun all suites and the cross-repo e2e	Gateway boots; every suite green as observed; e2e passes against sibling checkout; docs match the tree
1 - Shared foundations	Transport interface with local, relay, offline implementations; wire supabase stores into Manya-OS runtime; repair Manya gateway-to-fabric path; record package-sharing strategy; did:key identities replace throwaway uuids	Two processes exchange signed events through the relay; offline queue drains on reconnect; ledger persists across restart
2 - Cortex knowledge sharing	Implement KnowledgeRecord, registry service, and bridge dereference endpoint; extend both Cortex KnowledgeRegistries to honor foreign references; integrate grants	No-copy invariant test passes across two processes and identities, including a revoked-grant case
3 - Aruk security authority	Stand up Aruk service: identity registry, credential lifecycle, grant language, distributed revocation, trust evaluation, constitution PDP; harden bridge transport	End-to-end revocation propagates within the recorded budget; bridge fails closed without valid grants; device trust conditions grants
4 - Manya-Foundation build	Domain B system on shared foundations: registry, profiles, directory, announcements, communications, resources, onboarding; tier matrix as constitution policy; ego-graph surface via weave	Permission matrix enforced as tested policy; a member, a partner, and a visitor each see only their slice; every action ledgered
5 - Verification and hardening	Cross-domain e2e scenarios; load and resilience on transport and registry; security review of bridge and Aruk; documentation regenerated; release engineering aligned	All Section 10.1 verifications green; docs trustworthy; versioned release of both domains plus bridge

Table 9: Six phases with objectives and observable exit criteria.

10. Test Plan, Risks, and Open Decisions

Testing in this mandate has one purpose: to keep the difference between "the package exists" and "the system behaves" from ever quietly closing. Both repositories already demonstrate the failure mode - packages full of green tests that no runtime consumes. The verification matrix therefore spans levels, and every level runs in CI, not just on demand.

10.1 Verification matrix

Area	Method and standing rule	Phase
Unit integrity	All existing suites stay green; none may be deleted or skipped to make progress	0-5

Area	Method and standing rule	Phase
Runtime slice	Manya-OS runtime smoke (auth, governance, contracts, reason, memory, ledger) plus repaired Manya gateway e2e	0
Transport resilience	Signed-event exchange across two processes; kill and reconnect; offline drain; duplicate and replay rejection	1, 5
Persistence	Restart runtime; ledger and memory survive; migrations idempotent; RLS denies cross-tenant reads	1
No-copy invariant	Automated byte-level assertion that consumer stores hold references, not payloads; ledger shows grant on both sides	2
Authorization	Deny-by-default on every surface; expired and revoked grants fail; revocation propagation within budget	3
Visibility	Matrix-driven tests per tier for discovery, visibility, and access as three separate assertions	4
Cross-domain e2e	Scripted journeys: prospect to member; member shares knowledge with a Domain A intelligence; partner announcement reach	5

Table 10: Standing verification matrix across phases.

10.2 Risk register

Risk	L	I	Mitigation
Fork drift between duplicated packages reintroduces silent divergence	H	H	Recorded sharing strategy enforced in CI (5.3); divergence job fails the build
In-process assumptions are baked deeper than expected (sync fan-out, same-event-loop identity)	M	H	Transport behind existing API (5.1); local implementation keeps every current test green
Revocation latency becomes the weak edge of the security model	M	H	Propagation budget recorded and tested at Phase 3 exit; verifiers cache with expiry
Service-role credentials leak into client paths as Domain B grows	L	H	Server-side-only key discipline per DATABASE_BACKBONE.md; RLS on every member table
Scope creep: the Foundation platform drags the bridge into a product rewrite	M	M	Phase boundaries and exit criteria are the contract; changes require a decision-log entry
Documentation drift misleads future agents and stewards	H	M	Docs reconciliation is Phase 0 work; every phase ends with regenerated docs
Hardware-source stubs (Bluetooth, USB, camera) are mistaken for production capability	M	L	Stub status stated in docs; platform implementations are explicitly out of scope for this mandate

Table 11: Risk register (L = likelihood, I = impact).

10.3 Open decisions the agent must resolve with evidence

These decisions are deliberately left to the agent because the evidence to resolve them lives in the code and in measured behavior. Each resolution goes into a decision log entry - context, options considered, choice, and the evidence - so the Foundation can audit judgment, not just output.

Decision	Options in play	Resolution evidence required
Package sharing strategy	Source-of-truth repo with releases; synchronized subtree with CI guard; super-bundle split	Drift test results and CI wiring demonstrating enforcement
Transport protocol	WebSocket relay with JSON/CBOR; message broker; gRPC	Measured reconnect and throughput on the relay path against the resilience tests
Production database posture	Managed Supabase; self-hosted Postgres behind the same adapters	Migration run and RLS verification on the chosen target; backup and restore drill
Aruk deployment topology	Sidecar service per domain; standalone authority with cached verification	Failure-posture tests: grant continuity and denial behavior when Aruk is unreachable
Lycon interface finalization	Adopt the documented lycon:* channels and __lyconNative contract; renegotiate with the Lycon repository	Written interface agreement plus a contract test against the real host
Member data residency and privacy	POPIA-aligned (South Africa); GDPR-aligned for EU participants; both	A data map of Domain B stores with retention and export rules per class

Table 12: Open decisions log with required evidence.

One closing instruction to the agent: the repositories under your care are better than their documentation and worse than their reputation - that is unusual, and it is an advantage. Repair first, wire second, extend third, and let every ambitious mechanism in this brief - the bridge, the reference-based Cortex, Aruk, the Foundation domain - grow out of the tested primitives that already exist rather than replacing them. When the Foundation reviews your work, the evidence that will matter is running systems, green pipelines, and a decision log a steward can read.